

Signal data Generation

信号データの生成

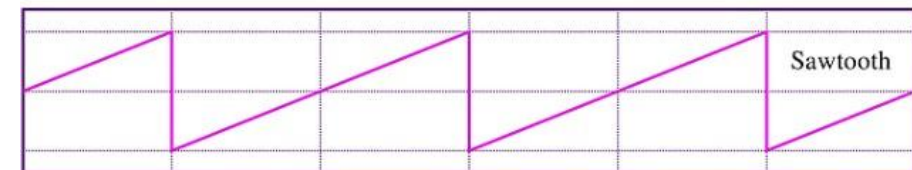
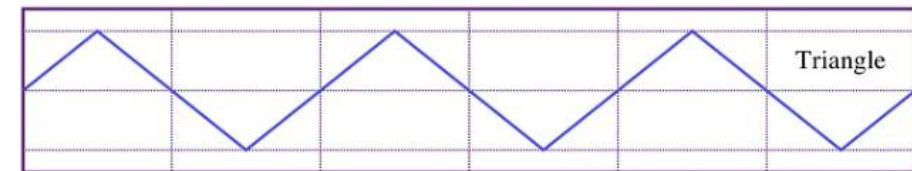
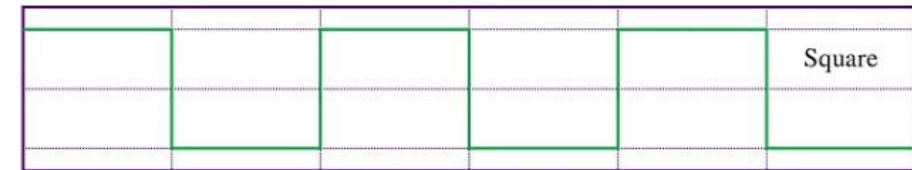
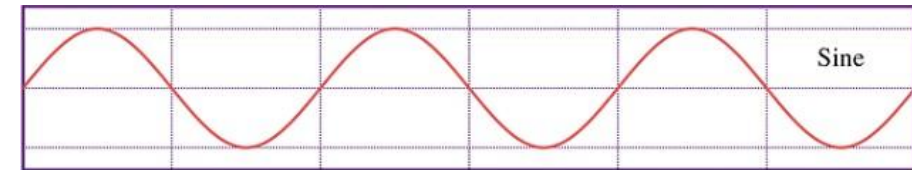
SILAA, Victor Alex
釧路工業高等専門学校
創造工学科(情報分野)
silaa@kushiro.kosen-ac.jp

波形 (waveform)

→ 波のグラフィカルな表現

graphical representation of waves.

Sine, Square, Triangle, Sawtooth
正弦波, 矩形波, 三角波, のこぎり波

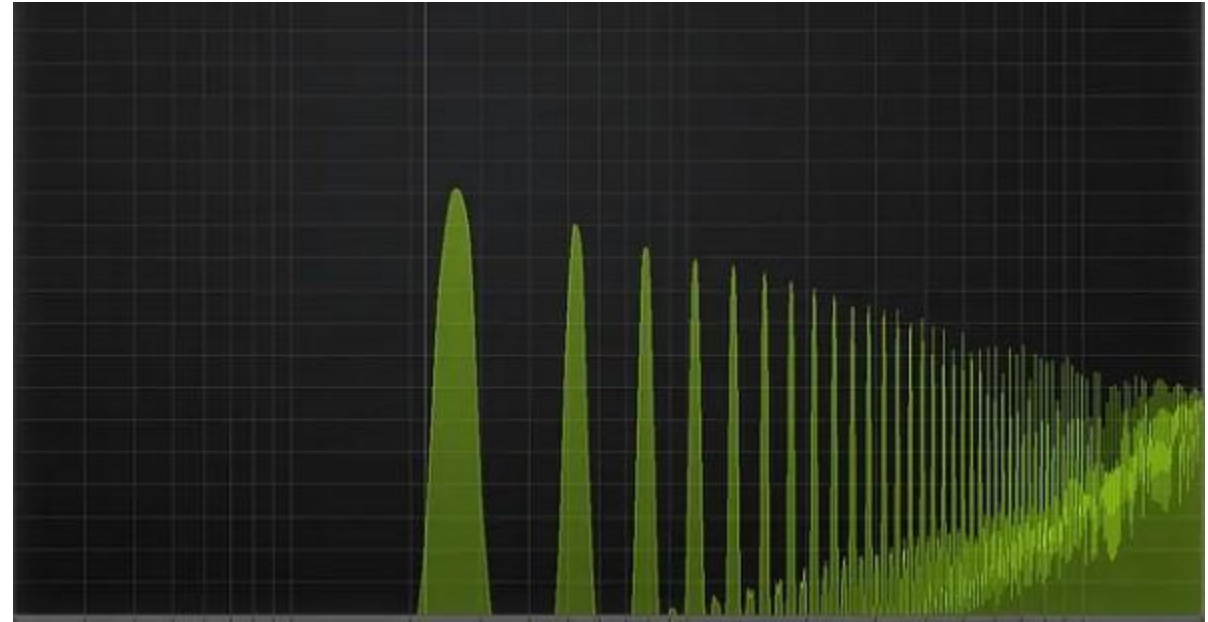


例えば,

(SAW)

→ 倍音が豊富

Rich in Harmonics. Most common waveform, used to make lots of sounds.



サンプルプログラム sample app wave.c

基本的な波形 wave.c → (Teams)

→ teams /wave.c

サンプルアプリ(wave.c)を使って基本波形のWAVファイルを作成する。

Use sample app (wave.c) to generate basic waveform WAV file

Audacity / paplay を使って波形と音色をチェックし、比較する。

Use audacity or paplay to check and compare waveform and tone

ディレクトリ → (~ / sp / pcm /) ソース → (wave.c)

サンプルプログラム sample app wave.c

- コンパイル :

- `$ cc wave.c -std=c99 -L. -lpcm -lm -o wave`

作業：信号の生成，波形と音色の確認・比較

Work : Generating signals, Checking and compare waveforms • tones

- `$ ./wave sin C > sin-C.wav; audacity sin-C.wav &`
- # 前回の sin.c と同じ，無表情な音色
- `$ ./wave tri C > tri-C.wav; ...` # sin とほぼ同じ or
木管楽器っぽい音色？
- `$ ./wave sqr C > sqr-C.wav; ...` # 木管楽器っぽい音
色？
- `$ ./wave saw C > saw-C.wav; ...` # 金管楽器 or 弦楽器
っぽい音色？

教科書の第3章などを参考にして

Reference : Textbook chapter 3

音響合成法 → 種類

 Types of sound synthesis (there are many more....)

Additive Synthesis → 加算合成

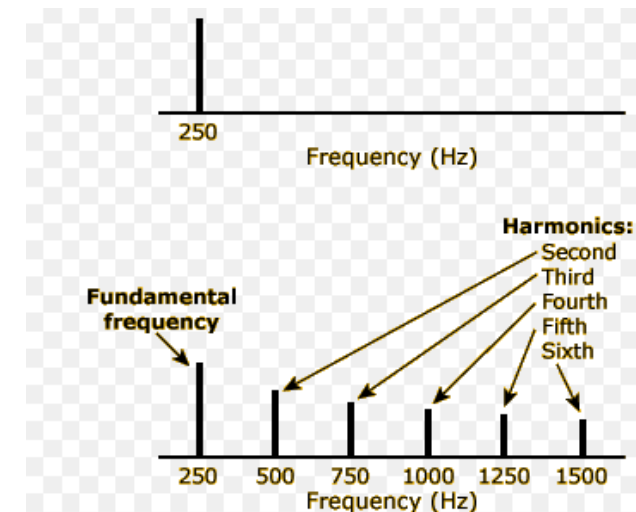
Subtractive Synthesis → 減算合成

Am synthesis → (am = amplitude modulation)

Fm synthesis → FM合成 (fm = frequency modulation)

理論：基本周波数 "fundamental frequency" の 整数倍の高調波周波数 "harmonic frequency" の正弦波を重ね合わせて、つまり、複数の正弦波の瞬時値を加算して、新しい波形を生成.

Generates a new waveform by superimposing sine waves with harmonic frequencies that are integer multiples of the fundamental frequency. I.e by adding instantaneous values of several sine waves.



→ 基本周波数の正弦波の音 : 基音 "fundamental tone" or 純音 "pure tone"

sine wave sound of fundamental frequency: Fundamental tone or pure tone

→ 整数倍周波数の正弦波の音 : 高調波 "harmonics" or 倍音 "overtone"

sine wave sound of integer multiple frequencies: harmonics or overtone

→ スペクトル展開 : 基音から N 倍音までの加重平均

Spectral expansion: weighted average from the fundamental to the Nth harmonic

基本的波形を複数の正弦波によって加算合成するプログラム synth.c を実装しよう。 Implement synth.c, a program that adds and combines basic waveforms with multiple sine waves

```
$ cp sin.c synth.c # 前回の sin.c を元に改造
$ vim synth.c
```

```
...
int main(...)
{
    // スペクトル分布
    #define N 50 // 例: 50倍音まで
    double a[N+1] = {}; // 0 ~ N 倍音の配合比率 (振幅スペクトル) の配列, 初期値0.0
    for (int k = 1; k <= N; k += 2) a[k] = 4.0/(M_PI*k); // 例: 矩形波 (奇数次高調波のみから成る)
    // 配列の要素番号は 0 ~ N. しかし, 要素ゼロは不使用.

    // スペクトルの正規化
    double a0 = 0.0; // 合計
    for (int k = 1; k <= N; k++) a0 += a[k];
    for (int k = 1; k <= N; k++) a[k] /= a0;

    /*
    // 音割れへの手抜きな対策. 振幅の合計を 1.0 としている.
    // 本来なら瞬時値の値域を ±1.0 とすべき

    // 実数による標本値の設定
    double *x0 = p->val[0];
    for (int i = 0; i < len; i++) {
        ...
        double v = 0.0;
        for (int k = 1; k <= N; k++) v += a[k]*sin(k*wt);
        x0[i] = v;
    }
    ...
}
```

modified based on the previous sin.c

Example : up to 50 harmonics

Ex. Square wave (consisting of odd harmonics only)

The array element numbers are 0 to N. However, element zero not used
Spectral Normalization

```
//sum
```

```
// A lazy solution to distortion. The sum of the amplitudes is set to 1.0
// The range of instantaneous values should be ±1.0
```

```
// set sample values using real numbers
```



```
$ cp sin.c synth.c # 前回の sin.c を元に改造
$ vim synth.c
```

```
...
int main(...)
{
    ...
    // スペクトル分布
    #define N 50 // 例: 50倍音まで
    double a[N+1] = {}; // 0 ~ N 倍音の配合比率 (振幅スペクトル) の配列, 初期値0.0
    for (int k = 1; k <= N; k += 2) a[k] = 4.0/(M_PI*k); // 例: 矩形波 (奇数次高調波のみから成る)
    // 配列の要素番号は 0 ~ N. しかし, 要素ゼロは不使用.

    // スペクトルの正規化
    /*
    double a0 = 0.0; // 合計
    for (int k = 1; k <= N; k++) a0 += a[k];
    for (int k = 1; k <= N; k++) a[k] /= a0;
    */

    // 音割れへの手抜きな対策. 振幅の合計を 1.0 としている.
    // 本来なら瞬時値の値域を±1.0 とすべき

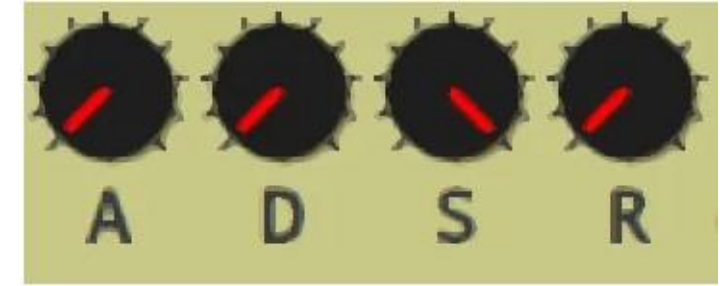
    // 実数による標本値の設定
    double *x0 = p->val[0];
    for (int i = 0; i < len; i++) {
        ...
        double v = 0.0;
        for (int k = 1; k <= N; k++) v += a[k]*sin(k*wt);
        x0[i] = v;
    }
    ...
}
```

コンパイル・実行 : Compile and Execute

```
$ CC ...  
$ ./synth C | paplay
```

とりあえず，矩形波のコード例を挙げました．鋸歯状波や三角波も作

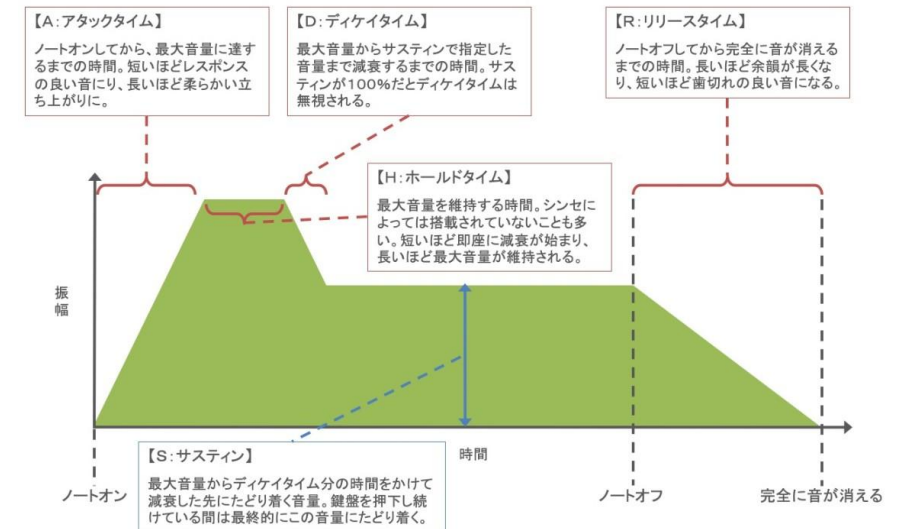
For now, I have provided a code example for a square wave. Lets's also create a sawtooth wave and a triangle wave.

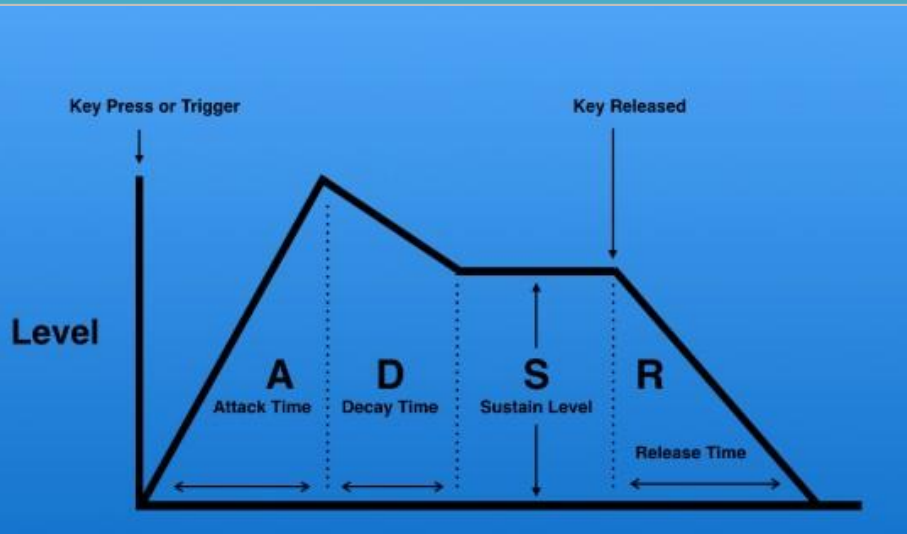


Envelope

ADSR: 一般的なシンセサイザ製品で利用される 包絡線形状モデリング用のパラメータ.

Parameter for envelope shape modeling used in general synthesizer products





A : アタックタイム (attack time) - 開始(無音)から最大音量までの起動時間

→ Startup time from start (silence) to maximum volume

D : ディケイタイム (decay time) - 最大音量から持続音量までの減衰時間

→ Decay time from maximum volume to sustained volume

S : サステインレベル (sustain level) - 持続音量

→ sustained volume

R : リリースタイム (release time) 終了(消音)までの減衰時間(残響 or 余韻の時間)

decay time – until end (mute)

包絡線形状の定義式の例

Example of envelope shape definition formula

$$e(t) = \begin{cases} 1 - \exp(-5t/A) & , \quad t = [0, A) \\ S + (1 - S) \exp\{-5(t - A)/D\} & , \quad t = [A, G) \\ S \exp\{-5(t - G)/R\} & , \quad t = [G, L) \end{cases}$$

実際のアナログシンセでは電気回路の過渡現象を利用していたので... この例では、それをシミュレートしているだけですよ。

The analog synthesizer used transients in electrical circuits. This example just simulating it.



:

G = L-R: ゲートタイム "gate time", 開始から持続終了までの時間

Time from start to end of duration → time from keyboard input on to off

L: デュレーション "duration" or "length of time", 開始から終了(消音)までの時間

Time from start to end of audio output.

プログラミング～実行：synth.c を改造し,何らかの包絡線形状を適用し,自分好みの楽器音を生み出そう.

Programming – Execution: Modify synth.c and apply some envelope shape to create your own instrument sounds.

```
$ vim synth.c
$ cc ...
...
```

とにかく，前半に強く，後半に弱まるように，強弱を付けるだけで充分でしょう．例えば，お手軽にピアノっぽくするには，D だけ考慮（ $A=S=R=0$ と大胆に近似）して， $e(t)=\{\cos(\pi t/L)+1\}/2$ とか $e(t)=1-t/L$ とかがオススメです

In any case, it should be enough to just add dynamics so that the first half is stronger and the second half is weaker. For example, to easily make it sound like a piano, I recommend only considering D (boldly approximating $A=S=R=0$), and using $e(t)=\{\cos(\pi t/L)+1\}/2$ or $e(t)=1-t/L$.

課題への準備： 自分好みの音色の **synth.c** が完成したら， 単音の **WAV** ファイルを用意しておこう

Preparations for the assignment : Once you have completed synth.c with your favorite tone, prepare a single-note WAV file

```
$ for $note in C D E F G A B C5
> do
> ./synth $note > $note.wav
> paplay $note.wav
> done
# ドレミファソラシド♭

$ ls *.wav
A.wav B.wav C.wav ...
# 単音のラ, シ, ド, ...のWAVファイル
```

\$note → note



Q1.複数の WAVファイルを入力し，音を連結（つなぎ合わせ）出力するプログラム seq.c を作成せよ.

Create a program, Seq.c, that accepts multiple WAV files and outputs a concatenation of sounds.

要するに,シーケンサー（自動演奏器）です.

In short, it is a sequencer (Automatic music player)

実行例:

Execution Example

```
$ ./seq C.wav D.wav E.wav > C-D-E.wav
$ paplay C-D-E.wav # 連続音のドレミ♫
```



ヒント : hint

処理手順 : processing steps

1. すべての入力ファイルを pcmLoad() - pcmLoad() all input files
2. pcmInit() で出力ファイル(データの容器)を作成 - Create an output file (data container) with pcmInit()
3. 各入力の全標本値を出力のデータ配列に追加コピー - Copies all sample values of each input into the output data array

【なお、個別の標本値ではなく、memcpy() で標本値の配列をコピーすれば秒殺です。バイナリデータ dat ではなく、実数データ val の方をコピーしてください。サンプルライブラリでは、出力時に val から dat を自動的に再構築するので、出力用の dat の内容は書き換えられてしまいますよ】 Note that it will be quicker if you use memcpy() to copy the array of sample values instead of individual sample values. Be sure to copy the real number data val, not the binary data dat. The sample library automatically reconstructs dat from val when outputting, so the contents of dat for output will be overwritten.

4. pcmWrite() でファイル出力 - output to file using pcmWrite()
5. 最後にpcmFin() - finally, pcmFin()



Q2.複数の WAVファイルを入力し,音を混合（重ね合わせ）出力するプログラム mix.c を作成せよ.

Create a program, mix.c that accepts multiple WAV files and outputs a mixture (superimposition) of sounds.

要するに, ミキサー(混合器)"mixer" ですね. 複数音で和音とか, 複数楽器で合奏.

In short, it is a Mixer (Automatic music player)

実行例:

Execution Example

```
$ ./seq C.wav D.wav E.wav > C-D-E.wav
$ paplay C-D-E.wav # 連続音のドレミ♫
```



ヒント: hint

処理手順: processing steps

1. WAV 構造体の配列を用意 – Prepare an array of WAV structures
2. pcmLoad() でファイル入力 – Input file with pcmLoad()
3. pcmInit() で出力ファイル(データの容器)を作成 – Create an output file (data container) with pcmInit()
4. 全入力の瞬時値を混合して出力の瞬時値を算出 – The instantaneous values of all inputs are mixed to calculate the instantaneous value of the output
5. 値域オーバーによる音割れ対策も必要 – Measures against distortion caused by exceeding the range of values are also necessary
6. pcmWrite() でファイル出力 – File output with pcmWrite()
7. 最後に pcmFin() – Finally, pcmFin()

Ex.

メロディ作成手順の例：（お辞儀の曲として知られるカデンツの演奏）

Example of melody creation procedure : (performance of cadenza known as a bowing piece)

```
$$ ./mix C5.wav E.wav G.wav > CEG.wav
$$ ./mix B.wav F.wav G.wav > BFG.wav
$$ ./seq CEG.wav BFG.wav CEG.wav | paplay
```

ドミソ
シファソ
ジャーン♪ジャーン♪ジャーン♪

提出方法： Teams

宛先： シラー ヴィクター アレックス_釧路

件名： sp-2704_学籍番号

提出期限： 5月11日（月） 17:00hrs